

Mastyf.ai × Jethur — Agentic AI PoC Checklist

Mastyf.ai — Agentic AI Pre-Deployment Security Assessment (PoC)

Customer: Jethur (internal validation)

Audience: Amil Shaji (Lead AI Engineer)

Prepared by: Mastyf.ai

Duration: ~2 weeks (adjust per app complexity)

Goal: Produce a repeatable, evidence-backed security assessment Jethur can use for release governance and future GRC integration.

A. Prerequisites (before Day 1)

Item	Owner	Notes
1–2 pilot applications identified (name, business function, risk tier)	Jethur	Prefer one MCP-native + one LangChain/OpenAI Agents app if available
Architecture brief: agent framework, MCP servers, upstream APIs, auth model	Jethur	Include staging URLs and who can access them
MCP client config(s) or equivalent tool manifest	Jethur	<code>mcp.json</code> , server definitions, tool list
Staging environment where agents can be pointed at Mastyf (not production-only)	Jethur	Fleet Hub / proxy URLs or middleware install
LLM for semantic depth (Ollama or approved cloud model) — optional but recommended	Joint	Needed for deep analysis / semantic policy
Data handling agreement for staging logs and findings	Joint	No production PII unless explicitly approved

Key scoping question: Do agents invoke tools via **MCP** or only **custom REST/API** agents? (Determines proxy vs. middleware path.)

B. Phase 1 — Inventory & static assessment (Days 1–3)

Step	Command / action	Pass criteria	Artifact
B1	Export/discover MCP configs for pilot apps	All servers and tools documented	Config inventory
B2	<code>mastyf-ai scan</code> (per config or <code>-a</code>)	Scores recorded; critical CVEs and secrets triaged	Scan report
B3	<code>mastyf-ai health</code>	Servers reachable in staging	Health summary
B4	<code>mastyf-ai threat-model --config <path></code>	STRIDE/LINDDUN threats reviewed with owners	Threat model (md/json)
B5	Review: transport (TLS/mTLS), auth on MCP ingress, secrets in config	No unmitigated critical gaps without acceptance	Risk register entries

C. Phase 2 — Staging runtime assessment (Days 4–8)

Step	Command / action	Pass criteria	Artifact
C1	Deploy Mastyf Fleet Hub or proxy with audit policy (<code>policy-audit.yaml</code>)	All pilot agent traffic flows through Mastyf	Proxy/fleet URLs
C2	Run realistic agent workflows (happy path)	Calls logged; no unexpected blocks on benign traffic	Audit sample
C3	Run adversarial prompts / tool abuse scenarios	Malicious patterns detected or explicitly accepted	Block/allow log
C4	Optional: switch to block policy in staging	Dangerous tool calls blocked without agent breakage on core flows	Policy tuning notes
C5	Dashboard review: SOC, Threat Lab, blocked-call patterns	Operators can triage and approve/reject rules	Screenshots / export
C6	MCP fuzz / red-team (dashboard API or harness)	Critical fuzz failures = 0 or documented exceptions	Fuzz summary

Optional (novel findings lane): Set `MASTYF_AI_VULN_DISCOVERY_ENABLED=true` with live tap in staging → triage findings in Vuln Discovery; use disclosure package export for any validated novel issues (Mastyf does not invent CVE IDs).

D. Phase 3 — Certification & governance evidence (Days 9–10)

Step	Command / action	Pass criteria	Artifact
D1	<code>mastyf-ai certify</code> (or dashboard certification)	Level $\geq$ agreed threshold (e.g. Silver+) or gap plan	Certification result
D2	Generate compliance evidence (<code>pnpm enterprise:compliance-evidence</code> or dashboard export)	Mapped to agreed frameworks	Evidence JSON/PDF
D3	Executive summary for GRC	Findings, mitigations, release recommendation	1–2 page summary

Suggested release gate (customize with Jethur):

- No unresolved **CRITICAL** findings
- No hardcoded secrets in agent/MCP config
- Certification / security score above threshold
- Threat-model high items mitigated or formally accepted
- Audit trail available for assessment period

E. Integration discovery (parallel — Mohammed Khasim; Days 5–10)

Item	Outcome
APIs to expose in Jethur UI (scan trigger, scores, findings, evidence download)	API shortlist
Tenant / customer isolation model	Architecture note
Deployment: Jethur SaaS vs. customer VPC	Deployment diagram
Workflow: "Assess before deploy" in GRC (trigger $\rightarrow$ evidence $\rightarrow$ sign-off)	Wireflow / user story

F. Deliverables at PoC close

1. **Technical assessment report** (per app): static scan, threat model, runtime findings, certification level
2. **Evidence pack** suitable for GRC attachment (compliance export + audit summary)
3. **Gap list** with severity, owner, and target date
4. **Integration recommendation** for Jethur platform (APIs, tenancy, MVP scope)
5. **Joint readout** (30 min) with Muhammed Shihabuddeen / Muntasir Mansoor — go/no-go for broader rollout

Contacts & support

Role	Name	Focus
Jethur — AI engineering	Amil Shaji	Run assessment, staging wiring
Jethur — integration	Mohammed Khasim	API / platform embed
Jethur — alignment	Muntasir Mansoor	Scope, commercial, timeline
Mastyf	Rudraneel Das	Playbook, technical support, partnership

Mastyf quick start (staging): `mastyf-ai start` → dashboard at `http://localhost:4000` → Security / Vuln Discovery workspaces.

Documentation: <https://github.com/mastyf-ai/mastyf.ai> · Real-world integration guide in repo `docs/REAL_WORLD_INTEGRATION.md`